SLiM Workshop Series #11: Callbacks III: `modifyChild()`

Copyright © 2020–2025 Benjamin C. Haller. All rights reserved.

1. Start a new model in SLiMgui

Run SLiMgui, and close any existing model windows. Press command-N to make a new model window. Adapt the default script to the following code, or create this script from scratch:

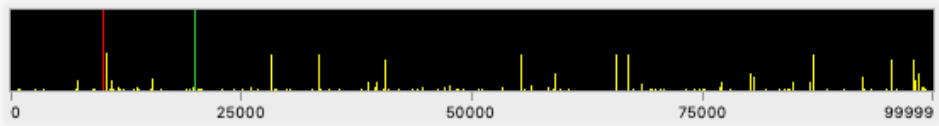
```
initialize() {
   initializeMutationRate(1e-7);
   initializeMutationType("m1", 0.5, "f", 0.0);

   initializeMutationType("m2", 0.5, "f", 0.5);  // mutation A
   m2.convertToSubstitution = F;
   m2.color = "red";

   initializeMutationType("m3", 0.5, "f", 0.5);  // mutation B
   m3.convertToSubstitution = F;
   m3.color = "#20D033";

   initializeGenomicElementType("g1", m1, 1.0);
   initializeGenomicElement(g1, 0, 99999);
   initializeRecombinationRate(1e-8);
}
1 late() {
   sim.addSubpop("p1", 500);
   sample(p1.haplosomes, 20).addNewDrawnMutation(m2, 10000);  // add A
   sample(p1.haplosomes, 20).addNewDrawnMutation(m3, 20000);  // add B
}
10000 early() { sim.simulationFinished(); }
```

This is a fairly simple one-subpopulation model with a chromosome that undergoes only neutral (`m1`) mutations. However, two other mutation types are defined, `m2` and `m3`, both of which are strongly beneficial (s=0.5). The model adds a single `m2` mutation (called "A") to `20` randomly chosen haplosomes, and a single `m3` mutation (called "B") to `20` randomly chosen haplosomes. It runs until tick `10000` and terminates.

We set two properties on those mutation types that we haven't encountered before: `convertToSubstitution` and `color`. The `convertToSubstitution` property, if set to `F`, prevents SLiM from replacing mutations of that mutation type with `Substitution` objects when they fix; the mutations will just continue segregating in the population, at a frequency of `1.0`, forever. This should be used with caution, since if large numbers of mutations are prevented from being substituted SLiM will run very slowly (since it will have to manage all of those mutations during offspring generation, fitness calculation, etc.). Here, however, we use it for the A and B mutations because we want them to interact *epistatically* – we want the fitness effect of A to depend upon whether B is also present, and *vice versa*. This epistatic interaction continues to happen even if either A or B fixes, so for simplicity we prevent them from being substituted when they fix. Neutral mutations will continue to be substituted upon fixation as usual, since we didn't change `m1`'s default `convertToSubstitution` value of `T`.

The `color` property sets the color used when displaying mutations of a given mutation type in SLiMgui. We set `m2` to a color of `"red"`, which, unsurprisingly, means it will display as red. SLiM knows many named colors; in fact, it uses the exact same named color list as R. A chart of the named color list in Eidos can be shown from the Help menu in SLiMgui. As in R, colors can also be specified with a hexadecimal RGB color; the color `"#20D033"` is a shade of green. We won't get into the weeds of hexadecimal, RGB, etc., at this point; the Eidos manual covers this topic.

If we run this model, the A and B mutations will almost always fix (since they are strongly beneficial and were introduced into 20 haplosomes initially), and we'll see a picture like this:



The A and B mutations, in red and green, can be clearly seen toward the left edge of the chromosome; both have fixed, but they have not been converted to `Substitution` objects because their mutation types have a `convertToSubstitution` value of `F`.

2. Add dominant lethal epistasis with a `modifyChild()` callback

We would like A and B to exhibit dominant lethal epistasis: any offspring that contains both mutations (whether homozygously or heterozygously) should experience developmental problems that prevent it from surviving. With a `modifyChild()` callback this is very easy to implement. This callback type is called once for each proposed offspring: every time that SLiM has generated an offspring individual and is about to add it to the target subpopulation, it calls any applicable `modifyChild()` callbacks to allow them to modify or veto the proposed child. Since we want to model a lethal developmental problem, we will veto proposed children that carry both A and B. Add the following code to your script (anywhere – at the end is fine):

```
modifyChild() {
   hasMutA = any(child.haplosomes.countOfMutationsOfType(m2) > 0);
   hasMutB = any(child.haplosomes.countOfMutationsOfType(m3) > 0);
   if (hasMutA & hasMutB)
      return F;
   return T;
}
```

First the callback determines whether the focal proposed child's haplosomes contain any `m2` or `m3` mutations. These tests are written in a general way, so that the code works even if multiple `m2` or `m3` mutations could exist at different chromosome positions; if the model can be hard-coded for a single `m2` and `m3` mutation at known positions, the `containsMarkerMutation()` method could be used instead and would be faster. We'll keep it general here, though. It is probably obvious, but the proposed child was passed to the callback through pseudo-parameter `child`.

The rest of the callback is exceedingly simple. If the proposed child contains both A and B mutations in its haplosomes, it returns `F`; this tells SLiM to suppress the generation of this proposed child. Otherwise, it returns `T`, telling SLiM to proceed with generation normally.

Recycle and run this model several times, and notice that in every run you get either the A mutation or the B mutation, but never both. The lethal epistasis completely prevents co-existence.

EXERCISE: Switch the callback to implement *recessive* lethal epistasis. In this case, the proposed child should be suppressed only if it is homozygous for both A and B. Homozygosity can be determined from the number of `m2` or `m3` mutations present in the proposed child's haplosomes.

In this exercise we're looking only at *lethal* epistasis, since a `modifyChild()` callback is well-suited to implementing that. It is straightforward to model non-lethal epistasis too, using a similar sort of strategy in a `mutationEffect()` callback. The SLiM manual has an example of this type of model.

3. Cultural inheritance

So far we have looked at a `modifyChild()` callback that returns `F` to veto a proposed child. It is also common to use `modifyChild()` callbacks to, as the name suggests, *modify* a proposed child without

vetoing it. We'll shift gears now completely, to look at such a model: a model of non-genetic cultural inheritance of a trait.

This model won't run until it's all built, but let's nevertheless build it one block at a time. The first component is the `initialize()` callback. Close any open SLiMgui windows, and then make a new window and modify its script to look like this:

```
initialize() {
   initializeMutationRate(1e-7);
   initializeMutationType("m1", 0.5, "f", 0.0);   // neutral
   initializeMutationType("m2", 0.5, "f", 0.1);   // lactase-promoting
   m2.convertToSubstitution = F;
   initializeGenomicElementType("g1", c(m1,m2), c(0.99,0.01));
   initializeGenomicElement(g1, 0, 99999);
   initializeRecombinationRate(1e-8);
}
```

There's a relatively infrequent mutation type, `m2`, that has a somewhat beneficial effect, and is prevented from being substituted at fixation using `convertToSubstitution` just as before. This mutation type promotes production of lactase (the enzyme that allows lactose, the common sugar in milk, to be digested). Next, create a new subpopulation:

```
1 early() {
   sim.addSubpop("p1", 1000);

   // start as mostly non-milk-drinkers
   p1.individuals.tagL0 = asLogical(rbinom(1000, 1, 0.01));
}
```

We use `addSubpop()` as usual. Then we do a vectorized property assignment into the `tagL0` property of all of the `p1` individuals. The `tagL0` property is like the `tag` (type `integer`) and `tagF` (type `float`) properties we saw before, but of type `logical`; it stores either `T` or `F`. There are five such `logical` properties on `Individual`, from `tagL0` to `tagL4`, for your own use; SLiM does not use them at all. They're very convenient for storing a *flag* – a single `logical` value. In this case, the flag stored in `tagL0` indicates whether the individual – for cultural reasons – does (`T`) or does not (`F`) drink milk as an adult. The values come from `rbinom()`, which generates draws from a binomial distribution; in this case, the values will be `0` about 99% of the time, whereas about 1% of values will be `1` (you can use the online help to read about `rbinom()`). We use `asLogical()` to convert from `integer` to `logical`; remember that in Eidos, as in many languages, `0` is considered `F` and `1` is considered `T`.

Next comes a `late()` event that runs in every tick, for purposes of output:

```
late() {
   // dark blue for non-milk-drinkers, light gray for milk-drinkers
   inds = p1.individuals;
   inds.color = ifelse(inds.tagL0, "lightgray", "darkblue");
   catn(sim.cycle + ": " + mean(inds.tagL0));
}
```

This assigns color values to individuals in each tick, based upon their `tagL0` value; if they are a milk-drinker (`tagL0` of `T`) they are colored light gray, whereas if they are a non-milk-drinker (`tagL0` of `F`) they are colored dark blue. We previously saw the use of `mutationType`'s `color` property; this is essentially the same, but sets the color of individuals. The vector of color names assigned into `inds.color` comes from the `ifelse()` function, which is essentially a vectorized `if-else` statement; see its documentation for further details. Finally, the event prints the fraction of milk-drinkers in each tick by taking the `mean()` of the `tagL0` values; remember, again, that `T` is `1` and `F` is `0` when used in numerical contexts like `mean()` and `sum()`. (Note that you can't run the model yet; you will get a "`tagL0` accessed before being set" error. We still need to set up `tag` values in the offspring.)

Next, add a very simple `mutationEffect()` callback:

```
mutationEffect(m2) {
   // beneficial for milk-drinkers, deleterious for non-milk-drinkers
   return individual.tagL0 ? effect else 0.95;
}
```

This makes the `m2` mutations slightly deleterious (with complete dominance) when found in non-milk-drinkers, by returning `0.95`. In milk-drinkers, on the other hand, it returns `effect` to use SLiM's default fitness effect for the mutation, which was set up to be somewhat beneficial. Since we've covered `mutationEffect()` callbacks already, this step is probably clear.

Define an end to the simulation with a termination event:

```
2500 early() { sim.simulationFinished(); }
```

And now, finally, we get to the interesting bit: using a `modifyChild()` callback to provide non-genetic inheritance of the cultural preference for milk-drinking. Add this to the script:

```
modifyChild() {
   // inherit culture from parents, with some deviation
   parentCulture = mean(c(parent1, parent2).tagL0);
   childCulture = rbinom(1, 1, 0.01 + 0.98 * parentCulture);
   child.tagL0 = (childCulture == 1);
   return T;
}
```

The parental values for the milk-drinking trait are obtained from their `tagL0` values (`parent1` and `parent2` are pseudo-parameters, of course, representing the first and second parents of the proposed child). Those values are averaged with `mean()`, and then `rbinom()` is used to draw a milk-drinking trait value for the child that is very heavily based upon the culture of its parents, but contains a tiny random element to provide a bit of stochasticity in cultural transmission. The child's chosen preference is recorded in its `tagL0` value (this time using `== 1` to convert from `integer` to `logical`, just for variety; we could instead use `asLogical()`, as we did before). Finally `T` is returned to tell SLiM to proceed with generation of the proposed child.

With that, all the script blocks are in place and we can run the model. You can visualize the shift from non-milk-drinking to milk-drinking in SLiMgui, through the coloring of individuals. The individuals rapidly shift from blue to gray, and the output from `catn()` shows the same pattern:

```
1: 0.021
2: 0.029
3: 0.032
4: 0.048
5: 0.051
...
100: 0.429

500: 0.956
...
1000: 0.984
...
```

This outcome is reproducible across runs, although there is variation in its speed. A question to briefly ponder: why does it happen so reliably? After all, the population starts out as almost entirely non-milk-drinkers, and the lactase-promoting mutations are deleterious in non-milk-drinkers. Shouldn't this mean that lactase-promoting mutations are weeded out, and therefore that there is no advantage to milk-drinking? Let's shift over to a graphical visualization to help us understand what's happening.

4. Plotting model output with LogFile

The first step to plotting model output is to collect the data. To do that, we will use a class we haven't seen before, `LogFile`. `LogFile` generates an output file that collects data produced by a simulation. The output is organized like a spreadsheet, into rows and columns; each column tracks the value of one metric over time, and each row represents the state of the model at one point in time.

Change the `1 early()` event to create a `LogFile` object by adding the lines in the middle here:

```
1 early() {
   sim.addSubpop("p1", 1000);

   // make a LogFile
   log = community.createLogFile("milk_drinkers.csv", logInterval=1);
   log.addTick();
   log.addCustomColumn("fraction", "mean(p1.individuals.tagL0);");

   // start as mostly non-milk-drinkers
   p1.individuals.tagL0 = asLogical(rbinom(1000, 1, 0.01));
}
```

The call to `community.createLogFile()` makes a new `LogFile` object and returns it. The first parameter is the filename; we use a ".csv" filename suffix because `LogFile` creates a file type called a CSV or "comma-separated value" file in which each value within a given row of the spreadsheet-like data is separated by a comma; we'll see that in a moment. The second parameter, `logInterval=1`, requests that this `LogFile` object should log new data every tick; we could pass `10` instead to log every ten ticks, for example.

The next two lines add columns to the "spreadsheet" of data being collected. The first adds a column showing the current tick; it will have a column name of `"tick"`, automatically. The second adds a custom column that we calculate ourselves; the first parameter to the `addCustomColumn()` call gives the column the name `"fraction"`. The second parameter provides a snippet of code, called a *lambda* in Eidos jargon, that is executed each time a new row is logged; it calls `mean()` to get the fraction of milk-drinkers in the population, just as we used to do with the line:

```
catn(sim.cycle + ": " + mean(inds.tagL0));
```

We're replacing that output with the `LogFile`, so you can remove that line, by the way.

It's important to understand the conceptual model here. The `LogFile` object is created and configured once, in `1 early()`. That object will exist for the rest of the simulation, kept by the `Community` object, and because `logInterval=1` was specified, it will log out a new row, with new values for the `tick` and `fraction` columns, in every tick. In the case of the `fraction` column, it will generate the value to be logged by executing, in every tick, the lambda script that was passed to `addCustomColumn()`; so `mean(p1.individuals.tagL0)` will execute each tick at the point when logging occurs (which is at the end of the tick, just before the tick and cycle counters are incremented).

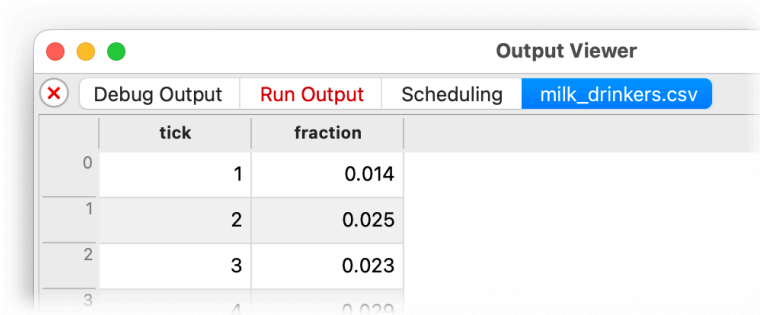
That's all the code changes needed. Recycle and run. The model should run as usual, without producing any visible output in SLiMgui, but a new file named `"milk_drinkers.csv"` should be on your desktop. If you open it in a text editor, you should see the output data like this:

```
tick,fraction
1,0.023
2,0.033
3,0.049
4,0.054
...
```

This is the CSV data, with values within each row separated by commas, and each row representing one time point at which data was logged. This is a standard file format; you can open it in Excel, or import it into a Google Sheet.

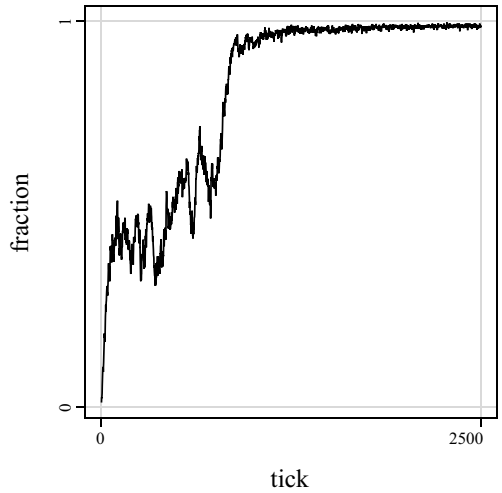
You can see this data directly in SLiMgui too. Click on the "show debugging output" button:

In the window that appears, select the rightmost tab, labeled "milk_drinkers.csv", and you should see something like this:



	tick	fraction
0	1	0.014
1	2	0.025
2	3	0.023

This lets you browse the logged data in SLiMgui, but it also lets you plot it! Control-click or right-click on the column header "fraction" to get a context menu with various options. Choose the command "Graph fraction ~ tick (line plot)" to get a plot like this:



EXERCISE: This plot can help us to understand the behavior of the model. Do your **BEEP!** first, now. Then think! One thing to think about, in understanding the behavior of this model, is what the model does if the mutation rate is set to `0.0`; do that, recycle and run, and make a new plot. What do you observe? Why does it happen? For another clue, look at the dynamics of the model (with the mutation rate restored to `1e-7`), as in the plot above. Does it transition from low to high milk-drinking in a linear fashion, or is there a sudden transition – perhaps triggered by…?

4. BEEP! With extra time, look at SLiM manual section 26.9, the reference doc for the `LogFile` class. Read the summary of the class itself, and then browse. It's an important and versatile tool.